

INDIVIDUAL PROJECT LOGBOOK

Project Title: Event-Driven Job Recruitment & Application Processing System (Event-Driven Architecture)

Student Name: Muhammad Harith Riezhan Bin Syanaz

Academic Role: Student

Group Number: Group 12

Team Member and Roles:

Name	Matric Number	Roles
MUHAMAD NUR IMAN HAFIZI BIN MOHD ADIB	S75659	Database Lead
MUHAMMAD HARITH RIEZHAN BIN SYANAZ	S75804	Backend Lead
MUHAMMAD SYUKRAN BIN MOHD ZAKRI	S75572	Frontend Lead

Weekly Progress

Phase 1: Project Proposal (Week 5)

Focuses on analyzing recruitment bottlenecks, setting system objectives, defining functional scopes, and compiling Deliverable 1.

Weekly Progress Metric	Details & Data Logs
Tasks Undertaken	Conducted initial team discussions to determine project boundaries and review architectural styles. Selected Topic C (Event-Driven Job Recruitment System) to overcome the processing lags and sequential bottlenecks typical of traditional helpdesks and platform setups. Partnered with teammates to outline system goals and compile the initial problem description for the proposal report.
Individual Contributions	Authored the design justification section detailing the choice of an Event-Driven Architecture (EDA) style. Mapped out the system's foundational components, illustrating how decoupling web handlers from heavy background processing layers optimizes system throughput. Formulated initial non-functional performance requirements, scalability metrics, and core platform objectives.
Challenges Faced	Clearly articulating and defending an event-driven design within a web application context without over-complicating the initial proposal scope.
Action Plan	Conducted targeted research into asynchronous event separation of concerns, focusing on event-driven dispatch patterns suited for multi-user platforms.

Phase 2: Architecture Design (Week 7)

Focuses on service decomposition, defining event boundaries, mapping relational schemas, and compiling Deliverable 2.

Weekly Progress Metric	Details & Data Logs
Tasks Undertaken	Led the overall system architecture design phase. Decomposed the recruitment system into isolated operational modules: Application, Screening, Interview, and Notification Services. Modeled and mapped out the core communication interaction boundaries across components using an event-driven approach. Modeled and finalized the full set of mandatory architecture diagrams for the design submission, including the Context, Component, Interaction, and Deployment views.
Individual Contributions	Hand-crafted the system deployment architecture view in VisualParadigm. Explicitly mapped out the boundaries between front-facing servlets (Producers) and internal async processors (Consumers). Wrote the complete Service and Component Identification section of the design document. Formulated the system's data ownership layout, establishing distinct relational schemas inside MySQL to prevent cross-service database locks and preserve strict module decoupling.
Challenges Faced	Determining the correct UML modeling elements and stereotypes for each asynchronous node and message consumer component while accurately mapping physical data boundaries across cloud environments.
Action Plan	Studied UML 2.0 structural stereotypes and deployment diagram conventions. Documented precise event schemas (e.g., ApplicationSubmittedEvent) to guarantee consistent message payloads across all decoupled modules.

Phase 3: Project Observations & Prototype Progress (Weeks 8 - 12)

Focuses on building the Java EE backend core, writing the async dispatcher, validating prototype flows, and passing execution checkpoints.

Project Observation 1 (Week 9)

Weekly Progress Metric	Details & Data Logs
Tasks Undertaken	Formally initialized the Java EE development stack. Built out core database integration configurations, data access objects (DAOs), and functional HTTP Servlets. Set up integration pathways between candidate entry actions and backend transaction handlers. Presented backend package architecture models and structure diagrams directly to the lecturer.
Individual Contributions	Programmed fundamental web controllers including UserController (handling authentication registration and login routing) and JobController (managing recruiter job publishing and discovery queries). Wrote custom relational SQL/JPQL queries to fetch specific applicant logs and catalog details securely.
Challenges Faced	Hit a critical null-pointer block during early form submissions where key request payload parameters sent from the client-side did not map correctly to the controller's backend parsing map.
Action Plan	Modified the backend controller execution flow to safely validate the inbound request stream using explicit key string lookups and typed data conversions. Verified the routing behavior using standalone Postman tests before rebuilding.

Prototype Progress (Week 11)

Weekly Progress Metric	Details & Data Logs
Tasks Undertaken	Completed the core system integration phase. Implemented processing services with automatic deadline-based status tracking. Developed the Recruiter Console dashboard views for viewing submitted applications and providing pipeline updates. Conducted end-to-end integration testing of all modules.
Individual Contributions	Coded the EventDispatcher logic and wired up independent background event consumers. Implemented automated processing routines within the ScreeningService to parse resume attributes as soon as an application event fires. Built the recruiter status workspace and evaluation screens. Conducted Postman testing across all service routing endpoints to verify correct event responses before the demo.
Challenges Faced	Encountered a data synchronization bug where changing a candidate's pipeline status inside the recruiter console would update the MySQL state, but the candidate's personal view would fail to reflect the change or refresh without a manual page reload.
Action Plan	Designed and added an event-driven refresh mechanism to the frontend interface that hooks directly into state changes, forcing the dashboard view to re-fetch clean, updated status data automatically.

Project Observation 2 (Week 12)

Weekly Progress Metric	Details & Data Logs
Tasks Undertaken	Conducted full system integration checks across all architecture layers. Tested the complete operational flow of the system from the frontend UI through the controller servlets, internal service layers, data repository modules, and the underlying MySQL database.
Individual Contributions	Added a new target data query endpoint in the Job Management controllers and implemented corresponding data retrieval filters. Added duplicate submission prevention in the application service logic using a stream filter check before persisting data.
Challenges Faced	Discovered a critical CORS configuration bug during integration testing: local calls worked perfectly, but deployed, non-local domain paths blocked data traffic between the client dashboard and the backend server endpoints.
Action Plan	Reconfigured the backend's header cross-origin access rules (WebConfig.java) to match the application's domain footprint precisely, immediately redeploying and verifying the connection.

Phase 4 & 5: Project Report & Final Demonstration (Weeks 13 - 14)

Focuses on final documentation reporting, quality attribute verification, and delivering the final live system defense.

Phase 4: Project Report (Week 13)

Weekly Progress Metric	Details & Data Logs
Tasks Undertaken	Led the final engineering report synthesis, compiling all system requirements, final architecture diagrams, and prototype code structures. Conducted a rigorous quality evaluation of our system's non-functional attributes (scalability and fault tolerance) under high load simulations. Finalized this individual progress logbook.
Individual Contributions	Wrote the comprehensive Architecture Style, Component Decomposition, and Technical Evaluation sections of our group's final project report. Conducted a peer-review of all technical documentation sections submitted by teammates to ensure structural consistency across the modules.
Challenges Faced	Coordinating report formatting, technical naming conventions, and class/service naming rules across all team members to present a uniform, cohesive technical document.
Action Plan	Organised a technical code-and-document review workshop where every module name, class definition, and database table was cross-referenced and synchronized across the final document.

Phase 5: Final Demonstration (Week 14)

Weekly Progress Metric	Details & Data Logs
Tasks Undertaken	Spearheaded the final software architecture demo preparation and project defense. Prepared the full-stack walkthrough covering the architecture design, backend package structure, asynchronous event flows, and live system paths. Ran system pre-checks to verify that database threads, local context paths, and the servlet context engine were fully active, responsive, and running optimally before evaluation.
Individual Contributions	Delivered the live architectural breakdown during the presentation. Conducted a full-stack code execution walkthrough for the lecturer, explaining the Event-Driven Patterns, backend package structure, and tracking a live request from initial form submission through the async dispatcher down to the database log updates.
Challenges Faced	Explaining the asynchronous relationship between event producers, the internal thread dispatcher engine, and downstream data access consumers clearly under strict presentation timing.
Action Plan	Utilized our visual interaction sequence diagrams, technology stack metrics, and data flow models to cleanly present our system's core decoupled benefits, ensuring a successful project completion.

Generative Ai Usage Record

Phase 1: Project Proposal (Week 5)

Weekly Progress Metric	Details & Data Logs
GenAI Entry 1 (Week 5)	Tool Used: Claude Purpose: Structuring a comprehensive problem description and matching objectives for an event-driven platform layout. Prompt Used: <i>"Help me break down the main operational bottlenecks of sequential recruitment systems and write three clear objectives for an event-driven application architecture."</i> Output Summary: Provided a targeted structural layout focusing on processing delays, applicant status transparency issues, and suggested architectural solutions using a decoupled event framework. How It Was Used: Used to structure the background problem statement and align the five foundational system objectives in our preliminary proposal document.

Phase 2: Architecture Design (Week 7)

Weekly Progress Metric	Details & Data Logs
GenAI Entry 2 (Week 7)	Tool Used: Claude Purpose: Mapping out asynchronous component interactions and event producer/consumer nodes for UML diagram verification. Prompt Used: <i>"What are the standard structural elements and message flows required in an interaction diagram for an event-driven recruitment system handling application submissions?"</i> Output Summary: Detailed the sequence of a candidate submission event firing from a front-facing web controller to a central thread dispatcher, which then passes data payloads asynchronously to downstream processing consumers. How It Was Used: Served as a design layout reference to help map our Interaction and Component diagrams manually within Visual Paradigm.
GenAI Entry 3 (Week 7)	Tool Used: Claude Purpose: Formulating decoupled database schema strategies and payload attributes for our event interface dictionary. Prompt Used: <i>"How should I structure the schema boundaries and JSON event payloads for independent modules like Application Service and Screening Service to avoid tight database coupling?"</i> Output Summary: Suggested a structured data template establishing independent lookup patterns and mapping distinct payload attributes (e.g., ApplicationSubmittedEvent) to maintain processing decoupling. How It Was Used: Formed the core foundation for Section 2.4 (Service and Component Identification) and guided our relational MySQL data isolation layout.

Phase 3: Project Observations & Prototype Progress (Weeks 8 - 12)

Weekly Progress Metric	Details & Data Logs
GenAI Entry 4 (Week 9)	Tool Used: Claude Purpose: Clarifying the relationship between front-facing HTTP Servlets and background worker threads within a modular monolithic layout. Prompt Used: <i>"Explain how a Java Servlet can pass a heavy file processing task to an asynchronous background thread pool using an internal Singleton event bus without blocking the user response."</i> Output Summary: Produced a technical explanation of thread-safe message passing, highlighting execution separation and request handling decoupling. How It Was Used: Shared among team members during Project Observation 1 to align package structures and verify our core asynchronous execution paths.
GenAI Entry 5 (Week 11)	Tool Used: Claude Purpose: Running a technical code optimization pass to catch loose data mappings or race conditions in our custom event engine before freezing our prototype features. Prompt Used: <i>"Analyze my internal event dispatch loop logic for potential concurrency blocks and verify that form data capture keys match exactly between the servlet request maps and the consumer data objects."</i> Output Summary: Flagged a structural risk where string-based parameter lookups in the web controller did not match the frontend input keys, and identified a missing null-guard boundary in our user validation handler. How It Was Used: Guided our manual codebase audit, allowing us to patch parameter mismatches in our Java controllers before pushing our functional prototype to GitHub.

Phase 4 & 5: Project Report & Final Demonstration (Weeks 13 - 14)

Weekly Progress Metric	Details & Data Logs
GenAI Entry 6 (Week 13)	Tool Used: Claude Purpose: Formulating load testing strategies to evaluate the scalability and stability of our internal thread pools. Prompt Used: <i>"Suggest a methodology using tools like Apache JMeter to simulate concurrent application submission spikes and measure the performance of an asynchronous internal thread dispatcher."</i> Output Summary: Outlined a simulation framework to measure processing response latencies under heavy concurrent workloads. How It Was Used: Used to run performance tests against our backend, generating empirical data to populate the Architecture Evaluation and NFR validation sections of the report.